

Dokumentacja projektu

SYSTEMY WBUDOWANE I MIKROPROCESORY

sem. 4 NSTAC 2025/2026

Konrad Rosinski 21533

Wstep

1.Opis projektu

Projekt przedstawia system alarmowy bazujący na mikrokontrolerze Arduino Uno R3. System monitoruje przestrzeń za pomocą trzech czujników ruchu PIR i w momencie wykrycia ruchu alarmuje to dzwiekowo. System jest kontrolowany z poziomu wyświetlacza zapewniając prosty w obsłudze interfejs sterowany trzema przyciskami i pokazujący opcje na wyświetlaczu. System zaprogramowany jest w środowisku Arduino w języku C.

2. Środowisko sprzętowe

2.1 Mikrokontroler:

- Arduino Uno R3

2.2 Wyjścia:

- 3x Czujnik ruchu PIR działające niezależnie od siebie, wykrywające ruch i wysyłające sygnał do mikrokontrolera przy wykryciu.

2.3 Wyjścia:

- Przetwornik piezoelektryczny, generujący dźwięk po wykryciu ruchu.
- Wyświetlacz LCD HD44780 16x2.

3. Wyprowadzenia

Pin	Nazwa zmiennej	I/O	Opis
D2	DB4	Wyjście	LCD - Linia danych
D3	DB5	Wyjście	LCD - Linia danych
D4	DB6	Wyjście	LCD - Linia danych
D5	DB7	Wyjście	LCD - Linia danych
D6	E	Wyjście	LCD - Sygnał zezwalający
D7	RS	Wyjście	LCD - Sygnał instrukcji
D8	sensorPins[0]	Wejście	Czujnik 1
D9	sensorPins[1]	Wejście	Czujnik 2
D10	sensorPins[2]	Wejście	Czujnik 3
D11	btn_up	Wejście	Przycisk w gore
D12	btn_ok	Wejście	Przycisk ok
D13	btn_down	Wejście	Przycisk w dol
A0	ALARM_OUTPUT_pin	Wyjście	Piezo
A1	sensorPins[3]	Wejście	Miejsce na czujnik

4. Działanie

4.1 - Stany systemu wyświetlane pod wyborem opcji

0 - Stan Rozbrojenia - Wyświetlany jako DISARMED. W tym stanie alarm nie będzie uruchamiał alarmu.

1 - Stan Uzbrojenia - Wyświetlany jako ARMED. Zmiana na ten status dzieje się po konczeniu odliczania po wybraniu opcji ubrojenia alarmu. Czujniki w tym stanie działają i naruszenie czujnika będzie skutkowało uruchomieniem alarmu

2 - Odliczanie do alarmu - wyświetla czas, po którym alarm będzie uruchomiony.

3 - Alarm uruchomiony - Stan ten wyświetla informacje o uruchomieniu alarmu (!!!ALARM!!!) oraz o nazwie czujnika, który wywołał alarm.

4. Naruszenie wejścia - Alarm się nie uruchamia, jednak wyświetlacz pokazuje stan "NARUSZONO WEJSC" i nazwę czujnika, który ten stan wywołał.

4.2 - Interfejs użytkownika - menu pozwalające na zarządzanie alarmem. Między opcjami menu można przechodzić wciskając przyciski góra i dół. Aby uruchomić widzianą na wyświetlaczu opcję, należy wcisnąć OK.

4.2.1 - >Uzbroj alarm - Po uruchomieniu zaczyna się odliczanie, po którym alarm zostanie uzbrojony i wejście z czujnika uruchomi alarm.

4.2.2 - >Rozbroj alarm - Po uruchomieniu alarm zostanie rozbrojony i przy wykryciu ruchu z czujnika, alarm się nie uruchomi.

4.2.3 - >Czujniki - Otwiera podmenu, w którym można zarządzać czujnikami. Po wybraniu tej opcji wyświetla się instrukcja - po wcisnięciu górnego przycisku przechodzi się do listy dodanych czujników, gdzie można sprawdzić dodane czujniki, na których pinach są podłączone oraz ich status (OK oraz NARUSZONY).

Po wcisnięciu dolnego przycisku można dodać czujnik, jego typ, oraz na którym pinie jest podłączony.

5. Kod i struktura

5.1 - Kod

```
int btn_up = 13;
int btn_ok = 12;
int btn_down = 11;

int ALARM_OUTPUT_pin = A0;

int E = 6;
int RS = 7;
int DB4 = 2;
int DB5 = 3;
int DB6 = 4;
int DB7 = 5;

int currentSystemState = 0;

int sensorPins[6] = {8, 9, 10, A1, 0, 0};
int sensorTypes[6] = {0, 0, 1, 2, 0, 0};
int sensorActive[6] = {1, 1, 1, 1, 0, 0};
int sensorCount = 4;

int currentMenuOption = 0;
int menuMode = 0;
unsigned long lastActivityTime = 0;
unsigned long inactivityTimeout = 5000;

int wizardStep = 0;
int wizardPin = 2;
int wizardType = 0;
int currentSensorListIndex = 0;

unsigned long exitCountdownStart = 0;
int exitSecondsLeft = 10;

unsigned long lastBlinkTime = 0;
int blinkState = 0;
int violatedSensorIndex = -1;

int lastUpState = HIGH;
int lastDownState = HIGH;
int lastOkState = HIGH;

int isUpPressed() {
    int state = digitalRead(btn_up);
    if (state == LOW && lastUpState == HIGH) { delay(50); lastUpState = LOW;
return 1; }
    if (state == HIGH) lastUpState = HIGH;
    return 0;
}

int isDownPressed() {
    int state = digitalRead(btn_down);
```

```

    if (state == LOW && lastDownState == HIGH) { delay(50); lastDownState =
LOW; return 1; }
    if (state == HIGH) lastDownState = HIGH;
    return 0;
}

int isOkPressed() {
    int state = digitalRead(btn_ok);
    if (state == LOW && lastOkState == HIGH) { delay(50); lastOkState = LOW;
return 1; }
    if (state == HIGH) lastOkState = HIGH;
    return 0;
}

void LCD_Write4Bits(unsigned char dtw) {
    digitalWrite(E, LOW);
    digitalWrite(DB4, (dtw & 0x01));
    digitalWrite(DB5, (dtw & 0x02));
    digitalWrite(DB6, (dtw & 0x04));
    digitalWrite(DB7, (dtw & 0x08));
    digitalWrite(E, HIGH);
    delayMicroseconds(2000);
    digitalWrite(E, LOW);
    delayMicroseconds(2000);
}

void LCD_WriteData4Bits(unsigned char dtw) {
    digitalWrite(RS, HIGH);
    LCD_Write4Bits(dtw >> 4);
    LCD_Write4Bits(dtw & 0x0F);
}

void LCD_WriteCommand4Bits(unsigned char dtw) {
    digitalWrite(RS, LOW);
    LCD_Write4Bits(dtw >> 4);
    LCD_Write4Bits(dtw & 0x0F);
}

void LCD_Init4bit() {
    digitalWrite(RS, LOW);
    digitalWrite(E, LOW);
    LCD_Write4Bits(0x03); delay(5);
    LCD_Write4Bits(0x03); delay(2);
    LCD_Write4Bits(0x03); delay(2);
    LCD_Write4Bits(0x02);
    LCD_WriteCommand4Bits(0x28);
    LCD_WriteCommand4Bits(0x0C);
    LCD_WriteCommand4Bits(0x01);
    LCD_WriteCommand4Bits(0x06);
}

void LCD_WriteText(char ttw[]) {
    int i = 0;
    while (ttw[i] != '\0') {
        LCD_WriteData4Bits(ttw[i]);
    }
}

```

```

        i++;
    }
}

void LCD_ClearRow(int row) {
    LCD_WriteCommand4Bits(row == 0 ? 0x80 : 0xC0);
    LCD_WriteText(" ");
}

void LCD_PrintAt(int row, char text[]) {
    LCD_ClearRow(row);
    LCD_WriteCommand4Bits(row == 0 ? 0x80 : 0xC0);
    LCD_WriteText(text);
}

void printSensorType(int type) {
    if (type == 0) LCD_WriteText("PIR");
    if (type == 1) LCD_WriteText("DRZWI");
    if (type == 2) LCD_WriteText("WIBRACJA");
}

void refreshMenuDisplay() {
    if (menuMode == 0) {
        if (currentMenuOption == 0) LCD_PrintAt(0, "> Uzbroy alarm ");
        if (currentMenuOption == 1) LCD_PrintAt(0, "> Rozbroj alarm ");
        if (currentMenuOption == 2) LCD_PrintAt(0, "> Czujniki ");
        if (currentMenuOption == 3) LCD_PrintAt(0, "> Ustawienia ");

        if (currentSystemState == 1) LCD_PrintAt(1, "Status: UZBROJONY");
        if (currentSystemState == 0) LCD_PrintAt(1, "Status: ROZBROJ.");
    }
    else if (menuMode == 1) {
        if (wizardStep == 0) {
            LCD_PrintAt(0, "Wybierz Pin wej:");
            LCD_ClearRow(1);
            LCD_WriteCommand4Bits(0xC0);
            LCD_WriteText(" Pin: D");
            char numBuf[4];
            itoa(wizardPin, numBuf, 10);
            LCD_WriteText(numBuf);
        }
        else if (wizardStep == 1) {
            LCD_PrintAt(0, "Wybierz Typ:");
            LCD_ClearRow(1);
            LCD_WriteCommand4Bits(0xC0);
            LCD_WriteText(" <");
            printSensorType(wizardType);
            LCD_WriteText(">");
        }
    }
    else if (menuMode == 2) {
        if (sensorCount == 0) {
            LCD_PrintAt(0, "Brak czujnikow ");
            LCD_PrintAt(1, "BACK [OK]");
        }
        else {
            LCD_ClearRow(0);

```

```

        LCD_WriteCommand4Bits(0x80);
        char numBuf[4];
        itoa(currentSensorListIndex + 1, numBuf, 10);
        LCD_WriteText(numBuf);
        LCD_WriteText(":Pin ");
        itoa(sensorPins[currentSensorListIndex], numBuf, 10);
        LCD_WriteText(numBuf);
        LCD_WriteText(" ");
        printSensorType(sensorTypes[currentSensorListIndex]);

        int isViolated = digitalRead(sensorPins[currentSensorListIndex]);
        if (isViolated == HIGH) {
            LCD_PrintAt(1, "Stan: NARUSZONY!");
        } else {
            LCD_PrintAt(1, "Stan: OK");
        }
    }
}

void setup() {
    pinMode(btn_up, INPUT_PULLUP);
    pinMode(btn_ok, INPUT_PULLUP);
    pinMode(btn_down, INPUT_PULLUP);

    pinMode(ALARM_OUTPUT_pin, OUTPUT);
    digitalWrite(ALARM_OUTPUT_pin, LOW);

    for (int i = 0; i < sensorCount; i++) {
        pinMode(sensorPins[i], INPUT_PULLUP);
    }

    pinMode(E, OUTPUT); pinMode(RS, OUTPUT);
    pinMode(DB4, OUTPUT); pinMode(DB5, OUTPUT);
    pinMode(DB6, OUTPUT); pinMode(DB7, OUTPUT);
    LCD_Init4bit();

    LCD_PrintAt(0, "SYSTEM ALARMOWY");
    LCD_PrintAt(1, "START...");
    delay(1500);

    lastActivityTime = millis();
    refreshMenuDisplay();
}

void loop() {
    unsigned long currentTime = millis();

    if (currentSystemState == 3) {
        if (currentTime - lastBlinkTime ≥ 250) {
            lastBlinkTime = currentTime;
            if (blinkState == 0) {
                blinkState = 1;
                tone(ALARM_OUTPUT_pin, 1000);
            } else {
                blinkState = 0;
            }
        }
    }
}

```



```

        noTone(ALARM_OUTPUT_pin);
    }
}

if (violatedSensorIndex  $\neq$  -1) {
    LCD_PrintAt(0, "!!! ALARM !!! ");
    LCD_ClearRow(1);
    LCD_WriteCommand4Bits(0xC0);
    LCD_WriteText("CZUJNIK: ");
    printSensorType(sensorTypes[violatedSensorIndex]);
}

if (isOkPressed() == 1) {
    noTone(ALARM_OUTPUT_pin);
    currentSystemState = 0;
    menuMode = 0;
    refreshMenuDisplay();
}
return;
}

int anySensorViolatedNow = 0;
int tempViolatedIdx = -1;

for (int i = 0; i < sensorCount; i++) {
    if (sensorActive[i] == 1 && digitalRead(sensorPins[i]) == HIGH) {
        anySensorViolatedNow = 1;
        tempViolatedIdx = i;
        break;
    }
}

if (anySensorViolatedNow == 1) {
    if (currentSystemState == 1) {
        currentSystemState = 3;
        violatedSensorIndex = tempViolatedIdx;
        lastBlinkTime = currentTime;
        blinkState = 1;
        tone(ALARM_OUTPUT_pin, 1000);
        return;
    } else if (currentSystemState == 0) {
        currentSystemState = 4;
        LCD_PrintAt(0, "NARUSZONO WEJSCIE");
        LCD_ClearRow(1);
        LCD_WriteCommand4Bits(0xC0);
        LCD_WriteText("Pin:");
        char numBuf[4];
        itoa(sensorPins[tempViolatedIdx], numBuf, 10);
        LCD_WriteText(numBuf);
        LCD_WriteText(" Typ:");
        printSensorType(sensorTypes[tempViolatedIdx]);
    }
} else {
    if (currentSystemState == 4) {
        currentSystemState = 0;
    }
}

```

```

        refreshMenuDisplay();
    }
}

if (currentSystemState == 2) {
    int elapsed = (currentTime - exitCountdownStart) / 1000;
    int remaining = exitSecondsLeft - elapsed;

    if (remaining ≥ 0) {
        char numBuf[4];
        LCD_PrintAt(0, "Uzbrajanie... ");
        LCD_ClearRow(1);
        LCD_WriteCommand4Bits(0xC0);
        LCD_WriteText("Pozostalo: ");
        itoa(remaining, numBuf, 10);
        LCD_WriteText(numBuf);
        LCD_WriteText("s");

        if ((currentTime - exitCountdownStart) % 1000 < 50) {
            tone(ALARM_OUTPUT_pin, 800, 50);
        }
    } else {
        currentSystemState = 1;
        refreshMenuDisplay();
    }

    if (isOkPressed() == 1) {
        currentSystemState = 0;
        refreshMenuDisplay();
    }
    return;
}

if (currentTime - lastActivityTime ≥ inactivityTimeout) {
    if (menuMode ≠ 0 || currentMenuOption ≠ 0) {
        currentMenuOption = 0;
        menuMode = 0;
        refreshMenuDisplay();
    }
}

int actionTaken = 0;

if (isUpPressed() == 1) {
    actionTaken = 1;
    if (menuMode == 0) {
        currentMenuOption--;
        if (currentMenuOption < 0) currentMenuOption = 3;
    }
    else if (menuMode == 1) {
        if (wizardStep == 0) { wizardPin++; if (wizardPin > 13) wizardPin =
2; }
        else if (wizardStep == 1) { wizardType++; if (wizardType > 2)
wizardType = 0; }
    }
    else if (menuMode == 2) {

```



```

        delay(1000);
        subMenuSelected = 1;
        break;
    }
    menuMode = 1;
    wizardStep = 0;
    wizardPin = 2;
    wizardType = 0;
    subMenuSelected = 1;
    break;
}
}
}
else if (currentMenuOption == 3) {
    LCD_PrintAt(0, "Info. o systemie");
    LCD_PrintAt(1, "System alarmow");
    delay(1500);
}
}
else if (menuMode == 1) {
    if (wizardStep == 0) {
        wizardStep = 1;
    } else if (wizardStep == 1) {
        sensorPins[sensorCount] = wizardPin;
        sensorTypes[sensorCount] = wizardType;
        sensorActive[sensorCount] = 1;
        pinMode(wizardPin, INPUT_PULLUP);
        sensorCount++;

        LCD_PrintAt(0, "ZAPISANO CZUJNIK");
        LCD_PrintAt(1, "SUKCES!");
        delay(1200);
        menuMode = 0;
    }
}
else if (menuMode == 2) {
    menuMode = 0;
}
}

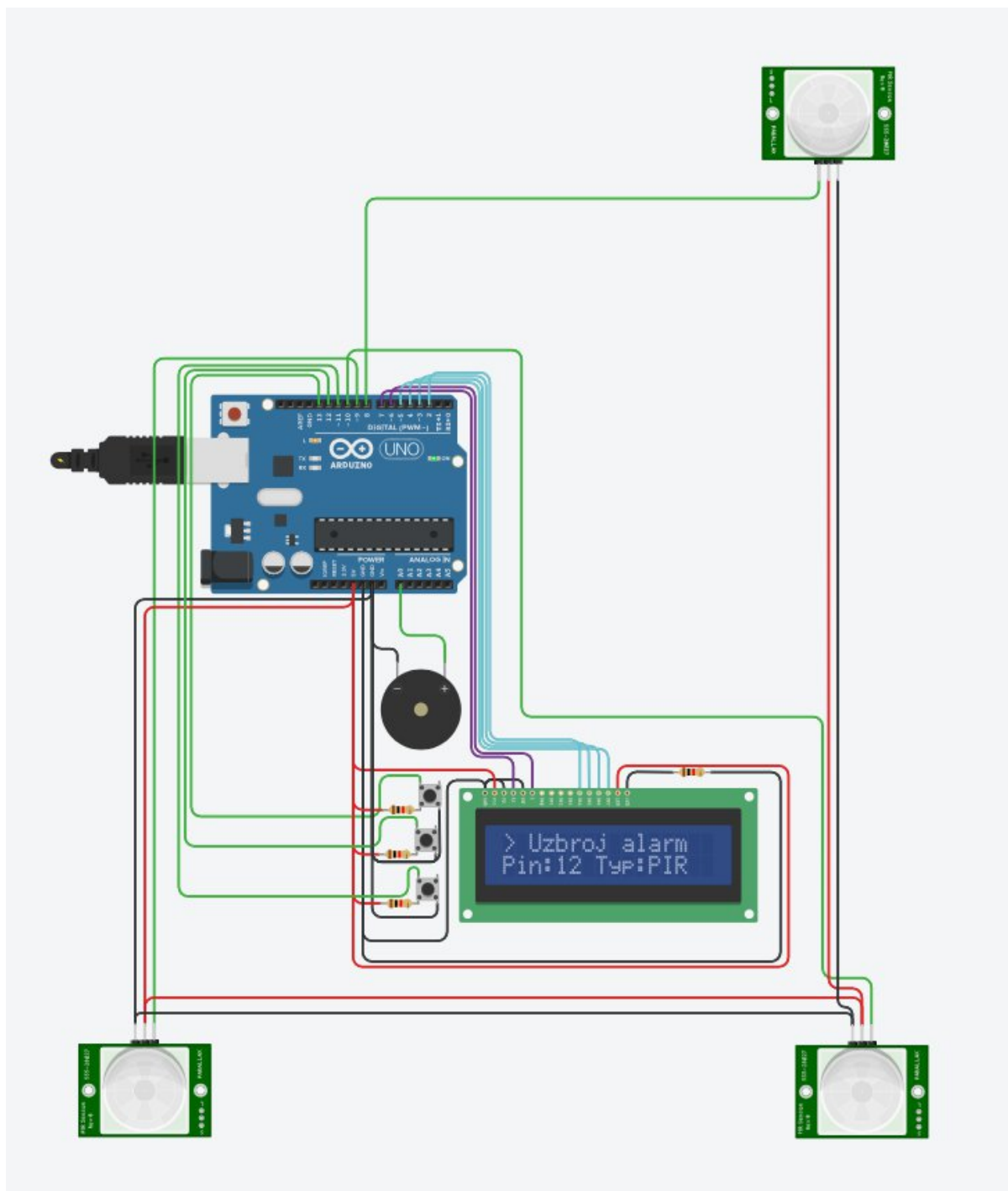
if (actionTaken == 1) {
    lastActivityTime = currentTime;
    refreshMenuDisplay();
}

if (menuMode == 2 && (currentTime % 300 < 30)) {
    refreshMenuDisplay();
}

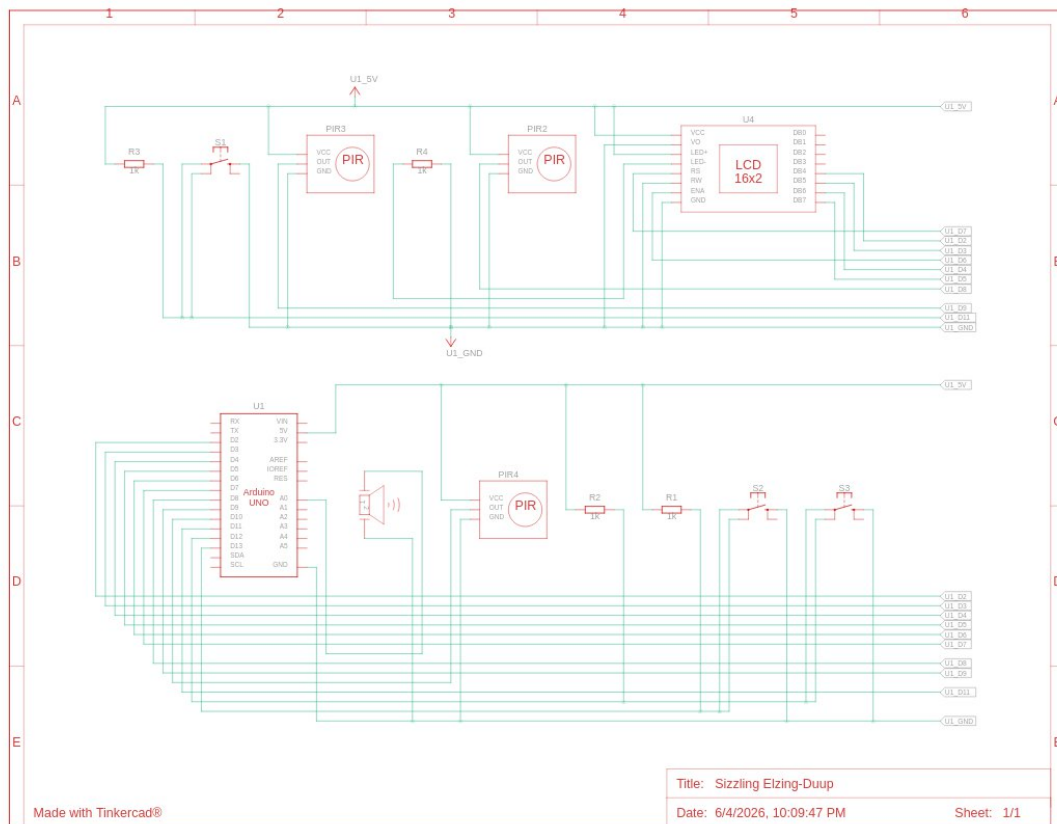
delay(20);
}

```

5.2.1 - Struktura fizyczna



5.2.2 - Schemat struktury



LINK TO TINKERCAD:

<https://www.tinkercad.com/things/9UKfo46M2YF-alarm>